```
scruffysGranTurismo.summary()
# wheels: phat
# dice: fluffy
```

The constructor could use the shorthand for arguments:

```
class GranTurismo
  constructor: (@options) ->
   summary: ->
     ("#{key}: #{value}" for key, value of @options).join "\n"
```

This is equivalent to the first version.

Exercise 5.3.3

Here are `Product` and `Camera` classes based on listing 5.4 with an alphabetical class method added to the `Camera` class:

```
class Product
# any implementation of Product

class Camera extends Product
  cameras = []
  @alphabetical = ->
    cameras.sort (a, b) -> a.name > b.name
  constructor: ->
    all.push @
    super
```

The `Camera` class can keep an array of all instances for `alphabetical` just as the `Product` class kept an array of products for `find`. The `Camera` constructor uses `super` to ensure the `Product` constructor is also invoked so that `Product.find` doesn't break.

Exercise 5.8.1

Applying some basic class techniques to the server-side application helps to increase clarity: